

LSTM and GRU Neural Network Model Training

June 6, 2026

1 Data Collection and Data Preprocessing

1.1 Importing the libraries

```
[ ]: import nltk
      nltk.download('gutenberg')
      from nltk.corpus import gutenberg
      import pandas as pd
```

```
[nltk_data] Downloading package gutenberg to /root/nltk_data...
[nltk_data]   Unzipping corpora/gutenberg.zip.
```

1.2 Fetching the dataset

```
[ ]: data = gutenberg.raw('shakespeare-hamlet.txt')
      with open('hamlet.txt', 'w') as file:
          file.write(data)
```

```
[ ]: import numpy as np
      from tensorflow.keras.preprocessing.text import Tokenizer
```

```
[ ]: from tensorflow.keras.preprocessing.sequence import pad_sequences
      from sklearn.model_selection import train_test_split
```

1.3 Converting the dataset into lower and save it

```
[ ]: with open('hamlet.txt', 'r') as file:
      text = file.read().lower()
```

1.4 Converting into tokens

```
[ ]: tokenizer = Tokenizer()
      tokenizer.fit_on_texts([text])
      total_words = len(tokenizer.word_index)+1
      total_words
```

```
[ ]: 4818
```

```
[ ]: # tokenizer.word_index
```

1.5 Creating the input sequence

```
[ ]: input_sequences=[]  
for line in text.split('\n'):  
    token_list = tokenizer.texts_to_sequences([line])[0]  
    for i in range(1, len(token_list)):  
        n_gram_sequence = token_list[:i+1]  
        input_sequences.append(n_gram_sequence)
```

```
[ ]: # input_sequences
```

1.6 Pad Sequencing

```
[ ]: max_sequence_len = max([len(x) for x in input_sequences])  
max_sequence_len
```

```
[ ]: 14
```

```
[ ]: input_sequences = np.array(pad_sequences(input_sequences,   
    ↪maxlen=max_sequence_len, padding='pre'))  
input_sequences
```

```
[ ]: array([[ 0,  0,  0, ...,  0,  1, 687],  
          [ 0,  0,  0, ...,  1, 687,  4],  
          [ 0,  0,  0, ..., 687,  4,  45],  
          ...,  
          [ 0,  0,  0, ...,  4,  45, 1047],  
          [ 0,  0,  0, ..., 45, 1047,  4],  
          [ 0,  0,  0, ..., 1047,  4, 193]], dtype=int32)
```

1.7 Creating prediction and labels

```
[ ]: import tensorflow as tf  
  
x, y = input_sequences[:, :-1], input_sequences[:, -1]
```

```
[ ]: y
```

```
[ ]: array([ 687,  4,  45, ..., 1047,  4, 193], dtype=int32)
```

```
[ ]: x
```

```
[ ]: array([[ 0,  0,  0, ...,  0,  0,  1],  
          [ 0,  0,  0, ...,  0,  1, 687],  
          [ 0,  0,  0, ...,  1, 687,  4],
```

```
...,
[ 0, 0, 0, ..., 687, 4, 45],
[ 0, 0, 0, ..., 4, 45, 1047],
[ 0, 0, 0, ..., 45, 1047, 4]], dtype=int32)
```

```
[ ]: y = tf.keras.utils.to_categorical(y, num_classes=total_words)
y
```

```
[ ]: array([[0., 0., 0., ..., 0., 0., 0.],
[0., 0., 0., ..., 0., 0., 0.],
[0., 0., 0., ..., 0., 0., 0.],
...,
[0., 0., 0., ..., 0., 0., 0.],
[0., 0., 0., ..., 0., 0., 0.],
[0., 0., 0., ..., 0., 0., 0.]])
```

1.8 Split the data into training and testing

```
[ ]: x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2,
↳random_state=42)
```

2 LSTM Neural Network Model Training

```
[40]: from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense, Dropout, Input, GRU
```

2.1 Define the Model (LSTM)

```
[ ]: model = Sequential()
model.add(Input(shape=(max_sequence_len-1,)))
model.add(Embedding(total_words, 100))
model.add(LSTM(150, return_sequences=True))
model.add(Dropout(0.2))
model.add(LSTM(100))
model.add(Dense(total_words, activation='softmax'))
```

2.2 Compile the model

```
[ ]: model.compile(loss = 'categorical_crossentropy', optimizer='adam',
↳metrics=['accuracy'])
model.summary()
```

Model: "sequential_2"

Layer (type)

Output Shape

Param #

embedding_1 (Embedding)	(None, 13, 100)	481,800
lstm_2 (LSTM)	(None, 13, 150)	150,600
dropout_1 (Dropout)	(None, 13, 150)	0
lstm_3 (LSTM)	(None, 100)	100,400
dense_1 (Dense)	(None, 4818)	486,618

Total params: 1,219,418 (4.65 MB)

Trainable params: 1,219,418 (4.65 MB)

Non-trainable params: 0 (0.00 B)

2.3 Train the model

```
[ ]: history = model.fit(x_train, y_train, epochs=100, validation_data=(x_test, ↵
    ↵y_test), verbose=1)
```

```
Epoch 1/100
644/644          10s 11ms/step -
accuracy: 0.0320 - loss: 6.9083 - val_accuracy: 0.0336 - val_loss: 6.6909
Epoch 2/100
644/644          8s 12ms/step -
accuracy: 0.0386 - loss: 6.4709 - val_accuracy: 0.0408 - val_loss: 6.7567
Epoch 3/100
644/644          11s 13ms/step -
accuracy: 0.0471 - loss: 6.3255 - val_accuracy: 0.0476 - val_loss: 6.7758
Epoch 4/100
644/644          6s 10ms/step -
accuracy: 0.0523 - loss: 6.1888 - val_accuracy: 0.0525 - val_loss: 6.8259
Epoch 5/100
644/644          7s 11ms/step -
accuracy: 0.0558 - loss: 6.0658 - val_accuracy: 0.0550 - val_loss: 6.8504
Epoch 6/100
644/644          6s 10ms/step -
accuracy: 0.0585 - loss: 5.9358 - val_accuracy: 0.0610 - val_loss: 6.9131
Epoch 7/100
644/644          7s 11ms/step -
accuracy: 0.0669 - loss: 5.8031 - val_accuracy: 0.0622 - val_loss: 6.9614
Epoch 8/100
644/644          6s 9ms/step -
```

accuracy: 0.0708 - loss: 5.6717 - val_accuracy: 0.0571 - val_loss: 6.9936
 Epoch 9/100
 644/644 8s 12ms/step -
 accuracy: 0.0791 - loss: 5.5375 - val_accuracy: 0.0659 - val_loss: 7.0666
 Epoch 10/100
 644/644 6s 9ms/step -
 accuracy: 0.0840 - loss: 5.4068 - val_accuracy: 0.0649 - val_loss: 7.1408
 Epoch 11/100
 644/644 8s 12ms/step -
 accuracy: 0.0903 - loss: 5.2830 - val_accuracy: 0.0657 - val_loss: 7.2095
 Epoch 12/100
 644/644 7s 11ms/step -
 accuracy: 0.0969 - loss: 5.1572 - val_accuracy: 0.0707 - val_loss: 7.3488
 Epoch 13/100
 644/644 6s 10ms/step -
 accuracy: 0.1033 - loss: 5.0275 - val_accuracy: 0.0641 - val_loss: 7.4511
 Epoch 14/100
 644/644 7s 11ms/step -
 accuracy: 0.1095 - loss: 4.8995 - val_accuracy: 0.0684 - val_loss: 7.5747
 Epoch 15/100
 644/644 6s 10ms/step -
 accuracy: 0.1146 - loss: 4.7776 - val_accuracy: 0.0643 - val_loss: 7.6720
 Epoch 16/100
 644/644 7s 11ms/step -
 accuracy: 0.1191 - loss: 4.6632 - val_accuracy: 0.0655 - val_loss: 7.8027
 Epoch 17/100
 644/644 6s 10ms/step -
 accuracy: 0.1264 - loss: 4.5504 - val_accuracy: 0.0635 - val_loss: 7.9189
 Epoch 18/100
 644/644 7s 11ms/step -
 accuracy: 0.1323 - loss: 4.4450 - val_accuracy: 0.0628 - val_loss: 8.0516
 Epoch 19/100
 644/644 6s 9ms/step -
 accuracy: 0.1411 - loss: 4.3440 - val_accuracy: 0.0639 - val_loss: 8.1915
 Epoch 20/100
 644/644 10s 10ms/step -
 accuracy: 0.1512 - loss: 4.2441 - val_accuracy: 0.0600 - val_loss: 8.3050
 Epoch 21/100
 644/644 7s 11ms/step -
 accuracy: 0.1630 - loss: 4.1546 - val_accuracy: 0.0583 - val_loss: 8.4250
 Epoch 22/100
 644/644 10s 11ms/step -
 accuracy: 0.1751 - loss: 4.0636 - val_accuracy: 0.0614 - val_loss: 8.5631
 Epoch 23/100
 644/644 6s 10ms/step -
 accuracy: 0.1909 - loss: 3.9700 - val_accuracy: 0.0618 - val_loss: 8.7059
 Epoch 24/100
 644/644 7s 11ms/step -

accuracy: 0.2006 - loss: 3.8934 - val_accuracy: 0.0565 - val_loss: 8.8356
 Epoch 25/100
 644/644 6s 10ms/step -
 accuracy: 0.2162 - loss: 3.8081 - val_accuracy: 0.0604 - val_loss: 8.9421
 Epoch 26/100
 644/644 7s 11ms/step -
 accuracy: 0.2290 - loss: 3.7376 - val_accuracy: 0.0563 - val_loss: 9.0665
 Epoch 27/100
 644/644 7s 10ms/step -
 accuracy: 0.2424 - loss: 3.6625 - val_accuracy: 0.0554 - val_loss: 9.1773
 Epoch 28/100
 644/644 7s 12ms/step -
 accuracy: 0.2532 - loss: 3.5928 - val_accuracy: 0.0571 - val_loss: 9.2865
 Epoch 29/100
 644/644 7s 11ms/step -
 accuracy: 0.2646 - loss: 3.5243 - val_accuracy: 0.0571 - val_loss: 9.3878
 Epoch 30/100
 644/644 8s 12ms/step -
 accuracy: 0.2764 - loss: 3.4635 - val_accuracy: 0.0540 - val_loss: 9.4861
 Epoch 31/100
 644/644 8s 12ms/step -
 accuracy: 0.2856 - loss: 3.4000 - val_accuracy: 0.0565 - val_loss: 9.5695
 Epoch 32/100
 644/644 7s 10ms/step -
 accuracy: 0.2968 - loss: 3.3448 - val_accuracy: 0.0521 - val_loss: 9.6910
 Epoch 33/100
 644/644 8s 12ms/step -
 accuracy: 0.3058 - loss: 3.2852 - val_accuracy: 0.0561 - val_loss: 9.7791
 Epoch 34/100
 644/644 7s 10ms/step -
 accuracy: 0.3162 - loss: 3.2293 - val_accuracy: 0.0542 - val_loss: 9.8468
 Epoch 35/100
 644/644 8s 12ms/step -
 accuracy: 0.3261 - loss: 3.1832 - val_accuracy: 0.0550 - val_loss: 9.9400
 Epoch 36/100
 644/644 9s 14ms/step -
 accuracy: 0.3347 - loss: 3.1261 - val_accuracy: 0.0546 - val_loss: 10.0249
 Epoch 37/100
 644/644 7s 10ms/step -
 accuracy: 0.3439 - loss: 3.0784 - val_accuracy: 0.0540 - val_loss: 10.1402
 Epoch 38/100
 644/644 7s 12ms/step -
 accuracy: 0.3510 - loss: 3.0305 - val_accuracy: 0.0540 - val_loss: 10.1934
 Epoch 39/100
 644/644 7s 10ms/step -
 accuracy: 0.3568 - loss: 2.9833 - val_accuracy: 0.0521 - val_loss: 10.2771
 Epoch 40/100
 644/644 8s 12ms/step -

accuracy: 0.3691 - loss: 2.9366 - val_accuracy: 0.0552 - val_loss: 10.3710
 Epoch 41/100
 644/644 10s 11ms/step -
 accuracy: 0.3762 - loss: 2.8924 - val_accuracy: 0.0517 - val_loss: 10.4284
 Epoch 42/100
 644/644 7s 10ms/step -
 accuracy: 0.3815 - loss: 2.8551 - val_accuracy: 0.0513 - val_loss: 10.5089
 Epoch 43/100
 644/644 8s 12ms/step -
 accuracy: 0.3920 - loss: 2.8111 - val_accuracy: 0.0536 - val_loss: 10.5924
 Epoch 44/100
 644/644 8s 12ms/step -
 accuracy: 0.4002 - loss: 2.7711 - val_accuracy: 0.0521 - val_loss: 10.6453
 Epoch 45/100
 644/644 7s 11ms/step -
 accuracy: 0.4060 - loss: 2.7305 - val_accuracy: 0.0499 - val_loss: 10.7250
 Epoch 46/100
 644/644 8s 12ms/step -
 accuracy: 0.4137 - loss: 2.6913 - val_accuracy: 0.0476 - val_loss: 10.8073
 Epoch 47/100
 644/644 10s 12ms/step -
 accuracy: 0.4203 - loss: 2.6569 - val_accuracy: 0.0493 - val_loss: 10.8619
 Epoch 48/100
 644/644 7s 10ms/step -
 accuracy: 0.4269 - loss: 2.6153 - val_accuracy: 0.0509 - val_loss: 10.9478
 Epoch 49/100
 644/644 7s 12ms/step -
 accuracy: 0.4334 - loss: 2.5807 - val_accuracy: 0.0490 - val_loss: 11.0058
 Epoch 50/100
 644/644 7s 10ms/step -
 accuracy: 0.4409 - loss: 2.5483 - val_accuracy: 0.0499 - val_loss: 11.0812
 Epoch 51/100
 644/644 7s 12ms/step -
 accuracy: 0.4488 - loss: 2.5106 - val_accuracy: 0.0505 - val_loss: 11.1627
 Epoch 52/100
 644/644 7s 10ms/step -
 accuracy: 0.4528 - loss: 2.4881 - val_accuracy: 0.0521 - val_loss: 11.1999
 Epoch 53/100
 644/644 8s 12ms/step -
 accuracy: 0.4600 - loss: 2.4525 - val_accuracy: 0.0507 - val_loss: 11.2671
 Epoch 54/100
 644/644 7s 11ms/step -
 accuracy: 0.4684 - loss: 2.4197 - val_accuracy: 0.0509 - val_loss: 11.3264
 Epoch 55/100
 644/644 7s 10ms/step -
 accuracy: 0.4736 - loss: 2.3853 - val_accuracy: 0.0523 - val_loss: 11.4333
 Epoch 56/100
 644/644 8s 12ms/step -

accuracy: 0.4767 - loss: 2.3593 - val_accuracy: 0.0507 - val_loss: 11.4545
 Epoch 57/100
 644/644 9s 10ms/step -
 accuracy: 0.4829 - loss: 2.3277 - val_accuracy: 0.0495 - val_loss: 11.4907
 Epoch 58/100
 644/644 7s 11ms/step -
 accuracy: 0.4864 - loss: 2.3026 - val_accuracy: 0.0490 - val_loss: 11.5820
 Epoch 59/100
 644/644 7s 11ms/step -
 accuracy: 0.4931 - loss: 2.2648 - val_accuracy: 0.0513 - val_loss: 11.6315
 Epoch 60/100
 644/644 7s 10ms/step -
 accuracy: 0.4990 - loss: 2.2431 - val_accuracy: 0.0482 - val_loss: 11.6883
 Epoch 61/100
 644/644 8s 12ms/step -
 accuracy: 0.5017 - loss: 2.2203 - val_accuracy: 0.0507 - val_loss: 11.7535
 Epoch 62/100
 644/644 9s 11ms/step -
 accuracy: 0.5104 - loss: 2.1897 - val_accuracy: 0.0478 - val_loss: 11.8193
 Epoch 63/100
 644/644 7s 11ms/step -
 accuracy: 0.5174 - loss: 2.1582 - val_accuracy: 0.0497 - val_loss: 11.8764
 Epoch 64/100
 644/644 11s 12ms/step -
 accuracy: 0.5226 - loss: 2.1401 - val_accuracy: 0.0501 - val_loss: 11.9037
 Epoch 65/100
 644/644 8s 13ms/step -
 accuracy: 0.5257 - loss: 2.1154 - val_accuracy: 0.0490 - val_loss: 11.9810
 Epoch 66/100
 644/644 7s 11ms/step -
 accuracy: 0.5306 - loss: 2.0850 - val_accuracy: 0.0495 - val_loss: 12.0285
 Epoch 67/100
 644/644 8s 12ms/step -
 accuracy: 0.5377 - loss: 2.0582 - val_accuracy: 0.0515 - val_loss: 12.1035
 Epoch 68/100
 644/644 7s 10ms/step -
 accuracy: 0.5420 - loss: 2.0353 - val_accuracy: 0.0519 - val_loss: 12.1287
 Epoch 69/100
 644/644 8s 12ms/step -
 accuracy: 0.5469 - loss: 2.0197 - val_accuracy: 0.0503 - val_loss: 12.2008
 Epoch 70/100
 644/644 8s 12ms/step -
 accuracy: 0.5515 - loss: 1.9923 - val_accuracy: 0.0503 - val_loss: 12.2306
 Epoch 71/100
 644/644 7s 11ms/step -
 accuracy: 0.5546 - loss: 1.9731 - val_accuracy: 0.0478 - val_loss: 12.3176
 Epoch 72/100
 644/644 8s 12ms/step -

accuracy: 0.5585 - loss: 1.9522 - val_accuracy: 0.0431 - val_loss: 12.3131
 Epoch 73/100
 644/644 6s 10ms/step -
 accuracy: 0.5645 - loss: 1.9286 - val_accuracy: 0.0447 - val_loss: 12.3884
 Epoch 74/100
 644/644 10s 10ms/step -
 accuracy: 0.5671 - loss: 1.9045 - val_accuracy: 0.0495 - val_loss: 12.4356
 Epoch 75/100
 644/644 8s 12ms/step -
 accuracy: 0.5746 - loss: 1.8819 - val_accuracy: 0.0464 - val_loss: 12.4886
 Epoch 76/100
 644/644 11s 13ms/step -
 accuracy: 0.5747 - loss: 1.8681 - val_accuracy: 0.0486 - val_loss: 12.5577
 Epoch 77/100
 644/644 7s 10ms/step -
 accuracy: 0.5821 - loss: 1.8382 - val_accuracy: 0.0480 - val_loss: 12.6047
 Epoch 78/100
 644/644 7s 11ms/step -
 accuracy: 0.5851 - loss: 1.8247 - val_accuracy: 0.0503 - val_loss: 12.6426
 Epoch 79/100
 644/644 7s 10ms/step -
 accuracy: 0.5907 - loss: 1.8034 - val_accuracy: 0.0486 - val_loss: 12.7050
 Epoch 80/100
 644/644 8s 13ms/step -
 accuracy: 0.5897 - loss: 1.7949 - val_accuracy: 0.0501 - val_loss: 12.7238
 Epoch 81/100
 644/644 7s 11ms/step -
 accuracy: 0.5973 - loss: 1.7735 - val_accuracy: 0.0474 - val_loss: 12.7754
 Epoch 82/100
 644/644 7s 10ms/step -
 accuracy: 0.6026 - loss: 1.7482 - val_accuracy: 0.0476 - val_loss: 12.8259
 Epoch 83/100
 644/644 8s 12ms/step -
 accuracy: 0.6057 - loss: 1.7253 - val_accuracy: 0.0460 - val_loss: 12.8720
 Epoch 84/100
 644/644 9s 10ms/step -
 accuracy: 0.6091 - loss: 1.7137 - val_accuracy: 0.0488 - val_loss: 12.9231
 Epoch 85/100
 644/644 8s 12ms/step -
 accuracy: 0.6146 - loss: 1.6939 - val_accuracy: 0.0490 - val_loss: 12.9842
 Epoch 86/100
 644/644 8s 12ms/step -
 accuracy: 0.6174 - loss: 1.6732 - val_accuracy: 0.0449 - val_loss: 13.0054
 Epoch 87/100
 644/644 7s 10ms/step -
 accuracy: 0.6207 - loss: 1.6618 - val_accuracy: 0.0460 - val_loss: 13.0412
 Epoch 88/100
 644/644 8s 12ms/step -

```

accuracy: 0.6227 - loss: 1.6420 - val_accuracy: 0.0488 - val_loss: 13.1121
Epoch 89/100
644/644          6s 10ms/step -
accuracy: 0.6275 - loss: 1.6274 - val_accuracy: 0.0492 - val_loss: 13.1334
Epoch 90/100
644/644          8s 12ms/step -
accuracy: 0.6315 - loss: 1.6117 - val_accuracy: 0.0472 - val_loss: 13.1500
Epoch 91/100
644/644          7s 11ms/step -
accuracy: 0.6348 - loss: 1.5954 - val_accuracy: 0.0460 - val_loss: 13.2258
Epoch 92/100
644/644          7s 11ms/step -
accuracy: 0.6385 - loss: 1.5870 - val_accuracy: 0.0459 - val_loss: 13.2800
Epoch 93/100
644/644          7s 11ms/step -
accuracy: 0.6411 - loss: 1.5664 - val_accuracy: 0.0459 - val_loss: 13.2927
Epoch 94/100
644/644          6s 10ms/step -
accuracy: 0.6430 - loss: 1.5512 - val_accuracy: 0.0488 - val_loss: 13.3339
Epoch 95/100
644/644          8s 12ms/step -
accuracy: 0.6490 - loss: 1.5303 - val_accuracy: 0.0509 - val_loss: 13.3978
Epoch 96/100
644/644          7s 11ms/step -
accuracy: 0.6509 - loss: 1.5250 - val_accuracy: 0.0472 - val_loss: 13.4566
Epoch 97/100
644/644          8s 12ms/step -
accuracy: 0.6528 - loss: 1.5094 - val_accuracy: 0.0468 - val_loss: 13.4560
Epoch 98/100
644/644         10s 11ms/step -
accuracy: 0.6565 - loss: 1.4940 - val_accuracy: 0.0459 - val_loss: 13.5436
Epoch 99/100
644/644         10s 12ms/step -
accuracy: 0.6574 - loss: 1.4850 - val_accuracy: 0.0499 - val_loss: 13.5503
Epoch 100/100
644/644          7s 11ms/step -
accuracy: 0.6612 - loss: 1.4700 - val_accuracy: 0.0497 - val_loss: 13.6082

```

2.4 Function to predict the next word with LSTM model

```

[ ]: def predict_next_word(seed_text, model, tokenizer, max_sequence_len):
    token_list = tokenizer.texts_to_sequences([seed_text])[0]
    if len(token_list) >= max_sequence_len:
        token_list = token_list[-(max_sequence_len-1):]
    token_list = pad_sequences([token_list], maxlen=max_sequence_len-1,
padding='pre')
    predicted = model.predict(token_list, verbose=0)

```

```

predicted_word_index = np.argmax(predicted, axis=1)
for word, index in tokenizer.word_index.items():
    if index == predicted_word_index:
        return word
return None

```

```
[ ]: input_text = 'to be or not to be'
```

```
[ ]: print(f"input text: {input_text}")
max_sequence_len = model.input_shape[1]+1
next_word = predict_next_word(input_text, model, tokenizer, max_sequence_len)
print(f"Next Predicted Word is: {next_word}")
```

input text: to be or not to be
Next Predicted Word is: made

2.5 Save the model

```
[33]: model.save('LSTM_model.h5')

import pickle
with open('tokenizer.pickle', 'wb') as handle:
    pickle.dump(tokenizer, handle, protocol=pickle.HIGHEST_PROTOCOL)
```

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

```
[39]: input_text = "Mar. haratio saies, 'tis but our"
print(f"input text: {input_text}")
max_sequence_len = model.input_shape[1]+1
next_word = predict_next_word(input_text, model, tokenizer, max_sequence_len)
print(f"Next Predicted Word is: {next_word}")
```

input text: Mar. haratio saies, 'tis but our
Next Predicted Word is: fantasie

2.6 Define the model (GRU)

```
[41]: model1 = Sequential()
model1.add(Input(shape=(max_sequence_len-1,)))
model1.add(Embedding(total_words, 100))
model1.add(GRU(150, return_sequences=True))
model1.add(Dropout(0.2))
model1.add(GRU(100))
model1.add(Dense(total_words, activation='softmax'))
```

```
[42]: model1.compile(loss = 'categorical_crossentropy', optimizer='adam',  
    ↪ metrics=['accuracy'])  
model1.summary()
```

Model: "sequential_3"

Layer (type)	Output Shape	Param #
embedding_2 (Embedding)	(None, 13, 100)	481,800
gru (GRU)	(None, 13, 150)	113,400
dropout_2 (Dropout)	(None, 13, 150)	0
gru_1 (GRU)	(None, 100)	75,600
dense_2 (Dense)	(None, 4818)	486,618

Total params: 1,157,418 (4.42 MB)

Trainable params: 1,157,418 (4.42 MB)

Non-trainable params: 0 (0.00 B)

```
[44]: history1 = model1.fit(x_train, y_train, epochs=100, validation_data=(x_test,  
    ↪ y_test), verbose=1)
```

```
Epoch 1/100  
644/644          7s 10ms/step -  
accuracy: 0.0487 - loss: 6.4162 - val_accuracy: 0.0563 - val_loss: 6.6728  
Epoch 2/100  
644/644          7s 12ms/step -  
accuracy: 0.0617 - loss: 6.1308 - val_accuracy: 0.0663 - val_loss: 6.6315  
Epoch 3/100  
644/644          6s 9ms/step -  
accuracy: 0.0769 - loss: 5.8338 - val_accuracy: 0.0727 - val_loss: 6.7082  
Epoch 4/100  
644/644          7s 11ms/step -  
accuracy: 0.0858 - loss: 5.5442 - val_accuracy: 0.0734 - val_loss: 6.7933  
Epoch 5/100  
644/644          6s 10ms/step -  
accuracy: 0.0977 - loss: 5.2615 - val_accuracy: 0.0734 - val_loss: 6.9202  
Epoch 6/100
```

644/644 7s 11ms/step -
accuracy: 0.1105 - loss: 4.9936 - val_accuracy: 0.0699 - val_loss: 7.0152
Epoch 7/100

644/644 7s 11ms/step -
accuracy: 0.1278 - loss: 4.7393 - val_accuracy: 0.0668 - val_loss: 7.1094
Epoch 8/100

644/644 6s 10ms/step -
accuracy: 0.1480 - loss: 4.5053 - val_accuracy: 0.0715 - val_loss: 7.2213
Epoch 9/100

644/644 7s 11ms/step -
accuracy: 0.1754 - loss: 4.2851 - val_accuracy: 0.0672 - val_loss: 7.3273
Epoch 10/100

644/644 6s 10ms/step -
accuracy: 0.2063 - loss: 4.0775 - val_accuracy: 0.0661 - val_loss: 7.4422
Epoch 11/100

644/644 8s 13ms/step -
accuracy: 0.2359 - loss: 3.8805 - val_accuracy: 0.0649 - val_loss: 7.5437
Epoch 12/100

644/644 9s 11ms/step -
accuracy: 0.2688 - loss: 3.6964 - val_accuracy: 0.0639 - val_loss: 7.6438
Epoch 13/100

644/644 6s 10ms/step -
accuracy: 0.2953 - loss: 3.5315 - val_accuracy: 0.0579 - val_loss: 7.7633
Epoch 14/100

644/644 7s 11ms/step -
accuracy: 0.3184 - loss: 3.3742 - val_accuracy: 0.0595 - val_loss: 7.8491
Epoch 15/100

644/644 6s 10ms/step -
accuracy: 0.3445 - loss: 3.2324 - val_accuracy: 0.0620 - val_loss: 7.9437
Epoch 16/100

644/644 7s 11ms/step -
accuracy: 0.3654 - loss: 3.0984 - val_accuracy: 0.0571 - val_loss: 8.0304
Epoch 17/100

644/644 6s 9ms/step -
accuracy: 0.3889 - loss: 2.9705 - val_accuracy: 0.0552 - val_loss: 8.1344
Epoch 18/100

644/644 7s 11ms/step -
accuracy: 0.4124 - loss: 2.8523 - val_accuracy: 0.0579 - val_loss: 8.2372
Epoch 19/100

644/644 7s 10ms/step -
accuracy: 0.4312 - loss: 2.7410 - val_accuracy: 0.0542 - val_loss: 8.3007
Epoch 20/100

644/644 7s 11ms/step -
accuracy: 0.4486 - loss: 2.6423 - val_accuracy: 0.0550 - val_loss: 8.3989
Epoch 21/100

644/644 7s 10ms/step -
accuracy: 0.4667 - loss: 2.5475 - val_accuracy: 0.0552 - val_loss: 8.4768
Epoch 22/100

644/644 10s 10ms/step -
accuracy: 0.4872 - loss: 2.4579 - val_accuracy: 0.0513 - val_loss: 8.5664
Epoch 23/100

644/644 7s 11ms/step -
accuracy: 0.5005 - loss: 2.3747 - val_accuracy: 0.0550 - val_loss: 8.6332
Epoch 24/100

644/644 6s 9ms/step -
accuracy: 0.5139 - loss: 2.2948 - val_accuracy: 0.0569 - val_loss: 8.7002
Epoch 25/100

644/644 7s 11ms/step -
accuracy: 0.5254 - loss: 2.2245 - val_accuracy: 0.0532 - val_loss: 8.7603
Epoch 26/100

644/644 6s 9ms/step -
accuracy: 0.5407 - loss: 2.1503 - val_accuracy: 0.0561 - val_loss: 8.8350
Epoch 27/100

644/644 7s 11ms/step -
accuracy: 0.5512 - loss: 2.0828 - val_accuracy: 0.0538 - val_loss: 8.9187
Epoch 28/100

644/644 7s 10ms/step -
accuracy: 0.5636 - loss: 2.0290 - val_accuracy: 0.0542 - val_loss: 8.9839
Epoch 29/100

644/644 10s 10ms/step -
accuracy: 0.5752 - loss: 1.9692 - val_accuracy: 0.0558 - val_loss: 9.0522
Epoch 30/100

644/644 7s 11ms/step -
accuracy: 0.5871 - loss: 1.9140 - val_accuracy: 0.0544 - val_loss: 9.1070
Epoch 31/100

644/644 6s 10ms/step -
accuracy: 0.5966 - loss: 1.8565 - val_accuracy: 0.0583 - val_loss: 9.1965
Epoch 32/100

644/644 7s 11ms/step -
accuracy: 0.6068 - loss: 1.8059 - val_accuracy: 0.0527 - val_loss: 9.2427
Epoch 33/100

644/644 11s 12ms/step -
accuracy: 0.6168 - loss: 1.7614 - val_accuracy: 0.0511 - val_loss: 9.2916
Epoch 34/100

644/644 6s 10ms/step -
accuracy: 0.6211 - loss: 1.7202 - val_accuracy: 0.0530 - val_loss: 9.3704
Epoch 35/100

644/644 7s 11ms/step -
accuracy: 0.6364 - loss: 1.6733 - val_accuracy: 0.0554 - val_loss: 9.4214
Epoch 36/100

644/644 10s 11ms/step -
accuracy: 0.6421 - loss: 1.6315 - val_accuracy: 0.0519 - val_loss: 9.5027
Epoch 37/100

644/644 6s 10ms/step -
accuracy: 0.6466 - loss: 1.6040 - val_accuracy: 0.0513 - val_loss: 9.5457
Epoch 38/100

644/644 10s 10ms/step -
accuracy: 0.6525 - loss: 1.5635 - val_accuracy: 0.0528 - val_loss: 9.6210
Epoch 39/100

644/644 10s 10ms/step -
accuracy: 0.6590 - loss: 1.5319 - val_accuracy: 0.0495 - val_loss: 9.6802
Epoch 40/100

644/644 7s 11ms/step -
accuracy: 0.6701 - loss: 1.4945 - val_accuracy: 0.0544 - val_loss: 9.7474
Epoch 41/100

644/644 6s 9ms/step -
accuracy: 0.6766 - loss: 1.4633 - val_accuracy: 0.0523 - val_loss: 9.7712
Epoch 42/100

644/644 7s 11ms/step -
accuracy: 0.6803 - loss: 1.4328 - val_accuracy: 0.0519 - val_loss: 9.8634
Epoch 43/100

644/644 6s 9ms/step -
accuracy: 0.6870 - loss: 1.4148 - val_accuracy: 0.0525 - val_loss: 9.8939
Epoch 44/100

644/644 10s 10ms/step -
accuracy: 0.6961 - loss: 1.3755 - val_accuracy: 0.0528 - val_loss: 9.9598
Epoch 45/100

644/644 7s 11ms/step -
accuracy: 0.6967 - loss: 1.3569 - val_accuracy: 0.0519 - val_loss: 10.0125
Epoch 46/100

644/644 9s 10ms/step -
accuracy: 0.7009 - loss: 1.3275 - val_accuracy: 0.0552 - val_loss: 10.0737
Epoch 47/100

644/644 7s 11ms/step -
accuracy: 0.7049 - loss: 1.3141 - val_accuracy: 0.0517 - val_loss: 10.1144
Epoch 48/100

644/644 7s 11ms/step -
accuracy: 0.7112 - loss: 1.2914 - val_accuracy: 0.0499 - val_loss: 10.1447
Epoch 49/100

644/644 6s 10ms/step -
accuracy: 0.7161 - loss: 1.2616 - val_accuracy: 0.0534 - val_loss: 10.2127
Epoch 50/100

644/644 7s 11ms/step -
accuracy: 0.7186 - loss: 1.2425 - val_accuracy: 0.0499 - val_loss: 10.2392
Epoch 51/100

644/644 6s 9ms/step -
accuracy: 0.7203 - loss: 1.2292 - val_accuracy: 0.0527 - val_loss: 10.3063
Epoch 52/100

644/644 10s 10ms/step -
accuracy: 0.7277 - loss: 1.2083 - val_accuracy: 0.0540 - val_loss: 10.3384
Epoch 53/100

644/644 7s 11ms/step -
accuracy: 0.7291 - loss: 1.1858 - val_accuracy: 0.0530 - val_loss: 10.4105
Epoch 54/100

644/644 6s 9ms/step -
accuracy: 0.7328 - loss: 1.1704 - val_accuracy: 0.0515 - val_loss: 10.4675
Epoch 55/100

644/644 7s 11ms/step -
accuracy: 0.7341 - loss: 1.1644 - val_accuracy: 0.0511 - val_loss: 10.4866
Epoch 56/100

644/644 10s 10ms/step -
accuracy: 0.7378 - loss: 1.1465 - val_accuracy: 0.0497 - val_loss: 10.5245
Epoch 57/100

644/644 7s 11ms/step -
accuracy: 0.7429 - loss: 1.1243 - val_accuracy: 0.0527 - val_loss: 10.5528
Epoch 58/100

644/644 7s 11ms/step -
accuracy: 0.7419 - loss: 1.1169 - val_accuracy: 0.0519 - val_loss: 10.6317
Epoch 59/100

644/644 6s 9ms/step -
accuracy: 0.7453 - loss: 1.1005 - val_accuracy: 0.0470 - val_loss: 10.6363
Epoch 60/100

644/644 7s 11ms/step -
accuracy: 0.7504 - loss: 1.0882 - val_accuracy: 0.0513 - val_loss: 10.6770
Epoch 61/100

644/644 6s 10ms/step -
accuracy: 0.7515 - loss: 1.0768 - val_accuracy: 0.0528 - val_loss: 10.7233
Epoch 62/100

644/644 7s 11ms/step -
accuracy: 0.7519 - loss: 1.0611 - val_accuracy: 0.0493 - val_loss: 10.7406
Epoch 63/100

644/644 6s 10ms/step -
accuracy: 0.7548 - loss: 1.0463 - val_accuracy: 0.0497 - val_loss: 10.8041
Epoch 64/100

644/644 7s 11ms/step -
accuracy: 0.7599 - loss: 1.0346 - val_accuracy: 0.0513 - val_loss: 10.8658
Epoch 65/100

644/644 10s 11ms/step -
accuracy: 0.7602 - loss: 1.0227 - val_accuracy: 0.0493 - val_loss: 10.8906
Epoch 66/100

644/644 6s 10ms/step -
accuracy: 0.7599 - loss: 1.0209 - val_accuracy: 0.0538 - val_loss: 10.9558
Epoch 67/100

644/644 10s 10ms/step -
accuracy: 0.7641 - loss: 1.0115 - val_accuracy: 0.0499 - val_loss: 10.9433
Epoch 68/100

644/644 7s 11ms/step -
accuracy: 0.7666 - loss: 1.0041 - val_accuracy: 0.0493 - val_loss: 10.9957
Epoch 69/100

644/644 6s 10ms/step -
accuracy: 0.7658 - loss: 0.9962 - val_accuracy: 0.0480 - val_loss: 11.0302
Epoch 70/100

644/644 11s 10ms/step -
accuracy: 0.7687 - loss: 0.9901 - val_accuracy: 0.0534 - val_loss: 11.0749
Epoch 71/100

644/644 7s 12ms/step -
accuracy: 0.7707 - loss: 0.9753 - val_accuracy: 0.0513 - val_loss: 11.1019
Epoch 72/100

644/644 6s 10ms/step -
accuracy: 0.7727 - loss: 0.9673 - val_accuracy: 0.0548 - val_loss: 11.1114
Epoch 73/100

644/644 11s 10ms/step -
accuracy: 0.7725 - loss: 0.9675 - val_accuracy: 0.0499 - val_loss: 11.1945
Epoch 74/100

644/644 7s 11ms/step -
accuracy: 0.7749 - loss: 0.9485 - val_accuracy: 0.0517 - val_loss: 11.1993
Epoch 75/100

644/644 6s 10ms/step -
accuracy: 0.7762 - loss: 0.9433 - val_accuracy: 0.0490 - val_loss: 11.2101
Epoch 76/100

644/644 7s 11ms/step -
accuracy: 0.7789 - loss: 0.9331 - val_accuracy: 0.0540 - val_loss: 11.2851
Epoch 77/100

644/644 6s 10ms/step -
accuracy: 0.7791 - loss: 0.9276 - val_accuracy: 0.0482 - val_loss: 11.3152
Epoch 78/100

644/644 7s 11ms/step -
accuracy: 0.7793 - loss: 0.9263 - val_accuracy: 0.0488 - val_loss: 11.3371
Epoch 79/100

644/644 10s 11ms/step -
accuracy: 0.7794 - loss: 0.9144 - val_accuracy: 0.0517 - val_loss: 11.3523
Epoch 80/100

644/644 9s 10ms/step -
accuracy: 0.7814 - loss: 0.9105 - val_accuracy: 0.0466 - val_loss: 11.3980
Epoch 81/100

644/644 7s 11ms/step -
accuracy: 0.7807 - loss: 0.9094 - val_accuracy: 0.0466 - val_loss: 11.4078
Epoch 82/100

644/644 7s 10ms/step -
accuracy: 0.7840 - loss: 0.9003 - val_accuracy: 0.0490 - val_loss: 11.4454
Epoch 83/100

644/644 7s 10ms/step -
accuracy: 0.7855 - loss: 0.8944 - val_accuracy: 0.0492 - val_loss: 11.4422
Epoch 84/100

644/644 7s 11ms/step -
accuracy: 0.7858 - loss: 0.8933 - val_accuracy: 0.0486 - val_loss: 11.5150
Epoch 85/100

644/644 6s 10ms/step -
accuracy: 0.7862 - loss: 0.8855 - val_accuracy: 0.0493 - val_loss: 11.5303
Epoch 86/100

644/644 7s 11ms/step -
accuracy: 0.7866 - loss: 0.8823 - val_accuracy: 0.0528 - val_loss: 11.5221
Epoch 87/100

644/644 10s 10ms/step -
accuracy: 0.7866 - loss: 0.8807 - val_accuracy: 0.0499 - val_loss: 11.5658
Epoch 88/100

644/644 7s 11ms/step -
accuracy: 0.7858 - loss: 0.8788 - val_accuracy: 0.0464 - val_loss: 11.5863
Epoch 89/100

644/644 7s 11ms/step -
accuracy: 0.7930 - loss: 0.8655 - val_accuracy: 0.0488 - val_loss: 11.6489
Epoch 90/100

644/644 6s 10ms/step -
accuracy: 0.7891 - loss: 0.8680 - val_accuracy: 0.0480 - val_loss: 11.6599
Epoch 91/100

644/644 7s 11ms/step -
accuracy: 0.7916 - loss: 0.8602 - val_accuracy: 0.0495 - val_loss: 11.7186
Epoch 92/100

644/644 6s 10ms/step -
accuracy: 0.7927 - loss: 0.8553 - val_accuracy: 0.0499 - val_loss: 11.7507
Epoch 93/100

644/644 7s 11ms/step -
accuracy: 0.7916 - loss: 0.8504 - val_accuracy: 0.0482 - val_loss: 11.7570
Epoch 94/100

644/644 6s 10ms/step -
accuracy: 0.7946 - loss: 0.8420 - val_accuracy: 0.0484 - val_loss: 11.7782
Epoch 95/100

644/644 8s 12ms/step -
accuracy: 0.7911 - loss: 0.8516 - val_accuracy: 0.0490 - val_loss: 11.8038
Epoch 96/100

644/644 10s 11ms/step -
accuracy: 0.7910 - loss: 0.8468 - val_accuracy: 0.0449 - val_loss: 11.8271
Epoch 97/100

644/644 10s 10ms/step -
accuracy: 0.7925 - loss: 0.8439 - val_accuracy: 0.0511 - val_loss: 11.8358
Epoch 98/100

644/644 7s 11ms/step -
accuracy: 0.7972 - loss: 0.8278 - val_accuracy: 0.0505 - val_loss: 11.8797
Epoch 99/100

644/644 7s 11ms/step -
accuracy: 0.7978 - loss: 0.8286 - val_accuracy: 0.0497 - val_loss: 11.8992
Epoch 100/100

644/644 6s 10ms/step -
accuracy: 0.7947 - loss: 0.8334 - val_accuracy: 0.0495 - val_loss: 11.9221

2.7 Function to predict the next word with LSTM model

```
[45]: def predict_next_word_GRU(seed_text, model1, tokenizer, max_sequence_len):
    token_list = tokenizer.texts_to_sequences([seed_text])[0]
    if len(token_list) >= max_sequence_len:
        token_list = token_list[-(max_sequence_len-1):]
    token_list = pad_sequences([token_list], maxlen=max_sequence_len-1,
    ↪padding='pre')
    predicted = model1.predict(token_list, verbose=0)
    predicted_word_index = np.argmax(predicted, axis=1)
    for word, index in tokenizer.word_index.items():
        if index == predicted_word_index:
            return word
    return None
```

```
[47]: input_text = 'to be or not to be'
print(f"input text: {input_text}")
max_sequence_len = model.input_shape[1]+1
next_word = predict_next_word_GRU(input_text, model, tokenizer,
    ↪max_sequence_len)
print(f"Next Predicted Word is: {next_word}")
```

input text: to be or not to be
Next Predicted Word is: made

```
[48]: model.save('GRU_model.h5')
```

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

```
[49]: import matplotlib.pyplot as plt

plt.figure(figsize=(12, 6))

# --- LSTM Accuracy ---

plt.plot(history.history['accuracy'], label='LSTM Train Accuracy',
    ↪color='blue', linestyle='-', linewidth=2)

plt.plot(history.history['val_accuracy'], label='LSTM Val Accuracy',
    ↪color='blue', linestyle='--', linewidth=2)

# --- GRU Accuracy ---
```

```

plt.plot(history1.history['accuracy'], label='GRU Train Accuracy',
         color='orange', linestyle='-', linewidth=2)

plt.plot(history1.history['val_accuracy'], label='GRU Val Accuracy',
         color='orange', linestyle='--', linewidth=2)

plt.title('Model Accuracy Comparison: LSTM vs. GRU', fontsize=16,
         fontweight='bold')
plt.xlabel('Epochs', fontsize=12)
plt.ylabel('Accuracy', fontsize=12)
plt.legend(loc='lower right', fontsize=10)
plt.grid(True, linestyle=':', alpha=0.7)

plt.show()

```

